

Kaggle Stroke Case

```
In [1]: import os
#import cv2
import time
import numpy as np
import pandas as pd
from sklearn.svm import SVC
import statsmodels.api as sm
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import precision_score, confusion_matrix, accuracy_score
from sklearn.model_selection import cross_val_score, train_test_split, cross_val_predict

import torch
from torch.autograd import Variable
from torchmetrics import Accuracy
import torch.nn.functional as F
import torch.nn as nn

targetDirectory = "C:\\Data\\Dynamic\\LECTURES\\Bachelor\\ML in der Praxis\\# IPYNB"
os.chdir(targetDirectory)#hier sucht Python nach csv Datei
data = pd.read_csv('StrokeData.csv', delimiter=';') # STROKE
# Überprüfen auf redundante und/oder NaN Daten
if data.duplicated().sum() > 0:
    data = data.drop_duplicates()
    print('Redundante Daten wurden entfernt.')
    print('-----')

if data.isna().any().sum() > 0:
    data = data.dropna()
    print('NaN-Daten wurden entfernt.')
    print('-----')
print(data.shape)
print("Working directory in use: ", os.getcwd())

print(torch.__version__)
```

```

cuda = True
if not torch.cuda.is_available():
    cuda = False
device = torch.device("cuda" if cuda else "cpu")

```

NaN-Daten wurden entfernt.

(4073, 11)

Working directory in use: C:\Data\Dynamic\LECTURES\Bachelor\ML in der Praxis\# IPYNB
1.12.1

```

In [2]: seed=2023
        #helper fct
        def set_seed_everywhere(seed, cuda):
            np.random.seed(seed)
            torch.manual_seed(seed)
            if cuda:
                torch.cuda.manual_seed_all(seed)

        set_seed_everywhere(seed, cuda)

```

```

In [3]: X = data[['gender', 'age', 'hypertension', 'heart_disease', 'ever_married', 'work_type',
                'Residence_type', 'avg_glucose_level', 'bmi', 'smoking_status']]
        y = data['stroke']
        # Datensatz in Trainings- und Testdaten aufteilen
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

        # Standardisierung
        scaler = StandardScaler()
        scaler.fit(X_train)
        X_train = scaler.fit_transform(X_train)
        X_test = scaler.transform(X_test)
        print("Data ready")

```

Data ready

```

In [4]: print(y_train.shape, X_train.shape)
        logreg = LogisticRegression()
        logreg.fit(X_train, y_train)

```

(3054,) (3054, 10)

Out[4]: ▾ LogisticRegression
LogisticRegression()

```
In [5]: # Vorhersagen für Testdaten treffen
log_pred = logreg.predict(X_test)
logit_model = sm.Logit(y_train, X_train)
result = logit_model.fit()

# Konfusionsmatrix
tn_log, fp_log, fn_log, tp_log = confusion_matrix(y_test, log_pred).ravel()
logreg_matrix = confusion_matrix(y_test, log_pred)

# FNR
fnr_log = fn_log / (fn_log + tp_log)

# CV Wert bei CV=10
logreg_cv = cross_val_score(logreg, X_train, y_train, cv=10).mean()

# CV und Vorhersagen für Testdaten treffen
log_cv_pred = cross_val_predict(logreg, X_train, y_train, cv=10)

# Konfusionsmatrix für Vorhersagen der Kreuzvalidierung berechnen
logreg_cv_matrix = confusion_matrix(y_train, log_cv_pred)

# FNR CV
tn_cv_log, fp_cv_log, fn_cv_log, tp_cv_log = logreg_cv_matrix.ravel()
fnr_cv_log = fn_cv_log / (fn_cv_log + tp_cv_log)
print(fnr_log)
```

Optimization terminated successfully.

Current function value: 0.686864

Iterations 4

1.0

```
In [6]: # Gütemaße ausgeben
print('-----')
print('Konfusionsmatrix Logit:\n' + f'{logreg_matrix}')
print('Accuracy Logit: %.3f' % accuracy_score(y_test, log_pred))
print('FNR Logit: %.3f' % fnr_log)
print('CV Konfusionsmatrix Logit:\n' + f'{logreg_cv_matrix}')
print('Cross-validation accuracy Logit: %.3f' % logreg_cv)
print('Cross-validation FNR Logit: %.3f' % fnr_cv_log)
```

```
print('Precision Logit: %.3f' % precision_score(y_test, log_pred, average='macro', zero_division=1))
print('Ergebnistapete Logit:' + f'{result.summary()}')
```

```
-----
Konfusionsmatrix Logit:
[[958  0]
 [ 61  0]]
Accuracy Logit: 0.940
FNR Logit: 1.000
CV Konfusionsmatrix Logit:
[[2907  0]
 [ 146  1]]
Cross-validation accuracy Logit: 0.952
Cross-validation FNR Logit: 0.993
Precision Logit: 0.970
Ergebnistapete Logit:
Logit Regression Results
=====
Dep. Variable:          stroke    No. Observations:          3054
Model:                  Logit    Df Residuals:              3044
Method:                  MLE     Df Model:                  9
Date:                    Thu, 13 Feb 2025    Pseudo R-squ.:          -2.559
Time:                    14:28:38    Log-Likelihood:          -2097.7
converged:                True    LL-Null:                  -589.37
Covariance Type:          nonrobust    LLR p-value:              1.000
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
x1              0.0101     0.037     0.277     0.782    -0.062     0.082
x2              0.1745     0.044     3.943     0.000     0.088     0.261
x3              0.0412     0.038     1.079     0.281    -0.034     0.116
x4              0.0411     0.039     1.062     0.288    -0.035     0.117
x5             -0.0249     0.041    -0.605     0.545    -0.106     0.056
x6             -0.0158     0.037    -0.429     0.668    -0.088     0.056
x7             -0.0077     0.036    -0.210     0.834    -0.079     0.064
x8              0.0782     0.039     2.030     0.042     0.003     0.154
x9             -0.0182     0.037    -0.487     0.626    -0.091     0.055
x10             0.0014     0.037     0.038     0.970    -0.071     0.074
=====
```

```
In [7]: #prep torch data
ser_y = pd.Series(y_train)
ser_y = ser_y.to_numpy()
print(ser_y.shape)
```

```
data_x = torch.from_numpy(X_train)
data_y = torch.from_numpy(y_train)
data_x = data_x.to(torch.float)
data_y = data_y.to(torch.long) #note Long
print(data_y.shape)
print(data_x.shape)
```

```
(3054,)
torch.Size([3054])
torch.Size([3054, 10])
```

```
In [8]: #PyTorch MLP
model = nn.Sequential(
    nn.Linear(10, 2),
    nn.Tanh(),
    nn.Linear(2, 2))
```

```
In [9]: optimizer = torch.optim.SGD(model.parameters(), lr=0.5)
loss_func = torch.nn.CrossEntropyLoss()
plt.ion()
```

```
Out[9]: <contextlib.ExitStack at 0x146a3aa0820>
```

```
In [10]: for t in range(100):
    prediction = model(data_x) # input x and predict based on x
    loss = loss_func(prediction, data_y) # must be (1. nn output, 2. target)
    optimizer.zero_grad() # clear gradients for next train
    loss.backward() # backpropagation, compute gradients
    optimizer.step() # apply gradients

    print("Epoch: %d, Loss: %f" % (t, float(loss)))
plt.ioff()
```

Epoch: 0, Loss: 0.408016
Epoch: 1, Loss: 0.324256
Epoch: 2, Loss: 0.279838
Epoch: 3, Loss: 0.252911
Epoch: 4, Loss: 0.235135
Epoch: 5, Loss: 0.222689
Epoch: 6, Loss: 0.213588
Epoch: 7, Loss: 0.206707
Epoch: 8, Loss: 0.201361
Epoch: 9, Loss: 0.197112
Epoch: 10, Loss: 0.193670
Epoch: 11, Loss: 0.190833
Epoch: 12, Loss: 0.188459
Epoch: 13, Loss: 0.186445
Epoch: 14, Loss: 0.184716
Epoch: 15, Loss: 0.183213
Epoch: 16, Loss: 0.181893
Epoch: 17, Loss: 0.180723
Epoch: 18, Loss: 0.179676
Epoch: 19, Loss: 0.178732
Epoch: 20, Loss: 0.177874
Epoch: 21, Loss: 0.177089
Epoch: 22, Loss: 0.176366
Epoch: 23, Loss: 0.175697
Epoch: 24, Loss: 0.175074
Epoch: 25, Loss: 0.174492
Epoch: 26, Loss: 0.173946
Epoch: 27, Loss: 0.173431
Epoch: 28, Loss: 0.172944
Epoch: 29, Loss: 0.172482
Epoch: 30, Loss: 0.172043
Epoch: 31, Loss: 0.171625
Epoch: 32, Loss: 0.171226
Epoch: 33, Loss: 0.170843
Epoch: 34, Loss: 0.170477
Epoch: 35, Loss: 0.170126
Epoch: 36, Loss: 0.169788
Epoch: 37, Loss: 0.169463
Epoch: 38, Loss: 0.169149
Epoch: 39, Loss: 0.168847
Epoch: 40, Loss: 0.168556
Epoch: 41, Loss: 0.168274
Epoch: 42, Loss: 0.168002
Epoch: 43, Loss: 0.167739

Epoch: 44, Loss: 0.167484
Epoch: 45, Loss: 0.167237
Epoch: 46, Loss: 0.166998
Epoch: 47, Loss: 0.166767
Epoch: 48, Loss: 0.166542
Epoch: 49, Loss: 0.166324
Epoch: 50, Loss: 0.166113
Epoch: 51, Loss: 0.165908
Epoch: 52, Loss: 0.165709
Epoch: 53, Loss: 0.165516
Epoch: 54, Loss: 0.165328
Epoch: 55, Loss: 0.165146
Epoch: 56, Loss: 0.164969
Epoch: 57, Loss: 0.164797
Epoch: 58, Loss: 0.164630
Epoch: 59, Loss: 0.164468
Epoch: 60, Loss: 0.164310
Epoch: 61, Loss: 0.164157
Epoch: 62, Loss: 0.164008
Epoch: 63, Loss: 0.163864
Epoch: 64, Loss: 0.163723
Epoch: 65, Loss: 0.163587
Epoch: 66, Loss: 0.163454
Epoch: 67, Loss: 0.163325
Epoch: 68, Loss: 0.163200
Epoch: 69, Loss: 0.163078
Epoch: 70, Loss: 0.162959
Epoch: 71, Loss: 0.162844
Epoch: 72, Loss: 0.162732
Epoch: 73, Loss: 0.162624
Epoch: 74, Loss: 0.162518
Epoch: 75, Loss: 0.162415
Epoch: 76, Loss: 0.162316
Epoch: 77, Loss: 0.162219
Epoch: 78, Loss: 0.162124
Epoch: 79, Loss: 0.162033
Epoch: 80, Loss: 0.161944
Epoch: 81, Loss: 0.161857
Epoch: 82, Loss: 0.161773
Epoch: 83, Loss: 0.161691
Epoch: 84, Loss: 0.161612
Epoch: 85, Loss: 0.161534
Epoch: 86, Loss: 0.161459
Epoch: 87, Loss: 0.161386

```
Epoch: 88, Loss: 0.161315  
Epoch: 89, Loss: 0.161246  
Epoch: 90, Loss: 0.161179  
Epoch: 91, Loss: 0.161113  
Epoch: 92, Loss: 0.161050  
Epoch: 93, Loss: 0.160988  
Epoch: 94, Loss: 0.160928  
Epoch: 95, Loss: 0.160870  
Epoch: 96, Loss: 0.160813  
Epoch: 97, Loss: 0.160757  
Epoch: 98, Loss: 0.160704  
Epoch: 99, Loss: 0.160651
```

```
Out[10]: <contextlib.ExitStack at 0x146b0596050>
```

```
In [11]: #Accuracy(prediction, data_y)  
out_probs = model(data_x).detach().numpy()  
out_classes = np.argmax(out_probs, axis=1)  
train_labels = y_train  
#print(train_labels[1:10])  
#print(out_classes[1:10])  
print(train_labels[3000:3010])  
print(out_classes[3000:3010])  
  
print("\n COMPARE")  
print('Accuracy Logit: %.3f' % accuracy_score(y_test, log_pred))  
print("Train Accuracy:", sum(out_classes == train_labels) / len(train_labels))  
print("\nXXX DONE XXX")
```



```
2007    0
609     0
4200    0
2880    0
3157    0
227     1
1908    0
1032    0
2893    0
3712    0
```

```
Name: stroke, dtype: int64
```

```
[0 0 0 0 0 0 0 0 0]
```

```
COMPARE
```

```
Accuracy Logit: 0.940
```

```
Train Accuracy: 0.9518664047151277
```

```
XXX DONE XXX
```

```
In [ ]:
```